

Doc. no.: P0487R0

Date: 2016-10-17

Audience: Library Working Group

Reply-to: Zhihao Yuan <zy at miator dot net>

Fixing operator>>(basic_istream&, CharT*) (LWG 2499)

Background

The [issue](#) was submitted with the following rationale: the most obvious use of this overload

```
std::cin >> buffer;
```

does not protect against buffer overflow, thus shares the same problem of stdio's `gets()`, which has been removed from both C11 and C++. So maybe we should remove this overload as well.

However, comparing it to `gets()` brings in some distortion here. More precisely, `scanf`'s "%s" is where this overload copies from. Both deal with formatted input, read “words”, and naive uses of them suffer from buffer overflow, plus both have ways to prevent this issue. For `scanf`, you can limit the field widths,

```
scanf("%20s %20s", a, b);
```

and the iostreams' version improved this practice by allowing programmatically passing the width:

```
cin >> setw(20) >> a;
```

The idea is as same as the "%.s" conversion specification in `printf`, while `scanf` doesn't support the asterisk ('*') arguments.

Discussion

What should we do to this library issue? People have raised the voices to deprecate or remove this overload. However, I want to mention that:

1. C is not deprecating or removing either "%s" or "%Ns" from `scanf`;
2. There are more than one existing legitimate uses of this overload. The users can pass the `.width()` argument to read unknown inputs, or read from streams with known contents and customized streams.

As shown as the proposed resolution to this issue, rather than deprecating or removing the whole overload, I try to:

1. Preserve some legitimate uses;
2. Protect the users against the bad uses.

More specifically, we can safely claim that when a width is not specified (`.width() == 0`), the user's intention is to read as if the length of the buffer is being passed. To an array type, the length is known at compile-time so that we can “fix” this for the user. However, due to implementability, unless we want to place additional preconditions on this function such as “Requires: `width() > 0` if the argument is of type `CharT*`”, all uses of passing a pointer to characters will have to be deprecated or removed.

In the following sections I provided two wordings, both adding the functionalities of taking array references, but one for deprecating the pointer arguments and one for removing. The deprecation option is nontrivial in certain ways.

The removal option may also be nontrivial to implement though, if an implementation wants to keep the ABI compatibility. The implementations are encouraged to use ABI tags or to guard the code to produce the old explicit specializations in the library binaries.

Wording for removal

This wording is relative to N4606.

Modify 27.7.2.2.3 [istream::extractors] as indicated:

```
template<class CharT, class traits, size_t N>
    basic_istream<CharT, traits>& operator>>(basic_istream<CharT, traits>& in,
        CharT* scharT (&s)[N]);
```

```
template<class traits, size_t N>
    basic_istream<char, traits>& operator>>(basic_istream<char, traits>& in,
                                           unsigned char* s unsigned char (&s)[N]);

template<class traits, size_t N>
    basic_istream<char, traits>& operator>>(basic_istream<char, traits>& in,
                                           signed char* s signed char (&s)[N]);
```

Effects: Behaves like a formatted input member (as described in 27.7.2.2.1 [istream.formatted.reqmts]) of `in`. After a sentry object is constructed, `operator>>` extracts characters and stores them into successive locations of an array whose first element is designated by `s`. If `width()` is greater than zero, `n` is `width()` `min(size_t(width()), N)`. Otherwise `n` is the number of elements of the largest array of `char_type` that can store a terminating `charT()`. `n` is the maximum number of characters stored.

Add a new compatibility item to C.4 [diff.cpp14]:

Clause 27: input/output library [diff.cpp14.input.output]

Change: Character array extraction only takes array types.

Rationale: Increase safety via preventing buffer overflow at compile time.

Effect on original feature: Valid C++ 2014 code may fail to compile in this International Standard:

```
auto p = new char[100];
std::cin >> std::setw(20) >> p;
```

Wording for deprecation

This wording is relative to N4606.

Modify 27.7.2.2.3 [istream::extractors] as indicated:

```
template<class charT, class traits, class arrayT>
    basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>& in,
                                           charT* arrayT&& s);

template<class traits, class arrayT>
    basic_istream<char, traits>& operator>>(basic_istream<char, traits>& in,
                                           unsigned char* arrayT&& s);

template<class traits>
    basic_istream<char, traits>& operator>>(basic_istream<char, traits>& in,
                                           signed char* s);
```

Let *A_T* denote `remove_reference_t<arrayT>`.

Remarks: The first form shall not participate in overload resolution unless `decay_t<arrayT>` is `charT*`. The second form shall not participate in overload resolution unless `decay_t<arrayT>` is `unsigned char*` OR `signed char*`.

Effects: Behaves like a formatted input member (as described in 27.7.2.2.1 [istream.formatted.reqmts]) of `in`. After a sentry object is constructed, `operator>>` extracts at most *K* characters and stores them into successive locations of an array whose first element is designated by `s`. If *A_T* is an array type in the form of `T[M]`, *K* = `min(size_t(width()), M)` if `width() > 0`, otherwise *K* = *M*. If *A_T* is a pointer type, *K* = `width()` if `width() > 0`, otherwise *K* if `width()` is greater than zero, `n` is `width()`. Otherwise `n` is the number of elements of the largest array of `char_type` that can store a terminating `charT()`. **The latter case is deprecated.** `n` is the maximum number of characters stored.

[Drafting note: Considering not putting nonexistent signatures in Annex D.]